Branch, Fail, Recover: A Spectral Theory of Reliable Agent Execution

Anonymous Author
Anonymous Institution

Abstract

LLM agents have gained widespread adoption, yet aggregate pass rates fail to capture how runtime branching, retries, and dynamic recovery govern system reliability and execution overhead. We address this by formalizing agent workflows as dependency graphs and analyzing their execution as multitype branching processes on unfolded runtime trees. In this model, the spectral radius of the conditional offspring matrix governs expected computational cost, while a Perron-weighted bound guarantees geometric tail decay. We then model recovery and identify a key tradeoff. Retries can reduce local failures but amplify total work, and when launched speculatively, they can push an otherwise subcritical workflow across the spectral threshold. An explicit resolvent bound weights subtask error probabilities by expected execution frequencies, providing a principled basis for recovery budget allocation. We also show that shared node difficulty induces heavy-tailed runtimes, explaining why average-case metrics fail to ensure tail safety. Controlled simulations corroborate these predictions, reproducing the spectral phase transition and showing that bounded retries lift workflow completion from 0.5224 to 0.9867 on a four-node benchmark. Live-agent evaluations on DataAgentBench (DAB) further exhibit consistent reliability versus cost trends. Together, these results provide practical guarantees for bounding and stabilizing agent runtimes under stochastic recovery.

Preliminary work. Under review by AISTATS 2027. Do not distribute.

1 INTRODUCTION

Large language model (LLM) agents are increasingly deployed across a broad spectrum of real world applications, spanning scientific data analysis, automated software engineering, and complex multi-tool and turn orchestration. At their core, modern agents build upon paradigms such as ReAct [3], interleaving reasoning traces with environment actions and empirical observations. As target tasks escalate in complexity, practical agentic systems naturally evolve beyond simple linear chains into composite workflows featuring parallel subtask decomposition, dynamic branching, and automated error-recovery routines. Because actions depend on runtime feedback, execution paths unfold on the fly. A single failure can trigger retries and new branches. This quickly multiplies the number of model calls. As a result, the total calls and runtime cannot be known in advance.

This uncertainty creates a resource planning problem. A serving system must store intermediate results, keep useful context in memory, and decide which adapters or cached states to retain. The expected number and types of future calls can inform these decisions. A bound on active branches can also guide memory reservations. A pass rate alone gives neither quantity. It can hide a small set of runs that consume much more work than the rest. Our goal is to establish the foundational stochastic workload theory providing closed-form, risk-bounded resource envelopes for any downstream serving system.

We study this problem through two linked models. A branching process describes how attempts create new calls and retries. A window graph describes how workers execute the resulting requests under memory, grouping, and loading constraints. The offspring matrix bounds expected work by task type. Its spectral radius gives a condition for finite expected work. A weighted exploration argument then bounds attempt counts and completion time even when outcomes depend on prior history. The analysis also reveals how validation order governs excess work. When a node

launches descendants before confirming its own output, each retry duplicates those descendants and compounds total cost. Validating before launch eliminates this amplification path entirely. We derive spectral thresholds for both orderings, quantify the resulting gap in expected work, and establish trace based conditions for estimating workload bounds in deployed systems. A shared difficulty model further explains why aggregate retry rates mask heavy runtime tails, proving that correlated failures across tasks inflate tail cost beyond mean field predictions. Together, these results supply concrete planning quantities including expected work bounds, retry budgets, and depth certificates, while leaving cache eviction policy as an orthogonal design choice.

We validate the theory across three complementary settings. Controlled simulations reproduce the predicted spectral phase transition and confirm the stabilizing effect of bounded retries. In live environments, paired executions on production model APIs demonstrate that validating nodes before descendant launch eliminates cascading downstream executions, directly verifying our excess work operator. Across broader workflows, evaluations on DataAgentBench show that our predicted reliability versus cost tradeoffs closely track empirical agent behavior on complex data tasks. Together, these results confirm that our theoretical bounds govern operational agent serving.

2 RELATED WORK

ReAct links reasoning to tool feedback [3]. Reflexion uses feedback from previous trials, while Self Refine iterates between generation, feedback, and revision [8, 9]. These methods motivate recovery with retained state. Tree of Thoughts explores several reasoning paths [10]. LLMCompiler organizes dependent tool calls for parallel execution [11]. Our question concerns the work created by repeated attempts and the effect of validating a parent before its descendants run.

Serving systems address the memory and scheduling cost of language model calls. PagedAttention manages attention cache memory, and SLoRA manages many adapters in shared serving [12, 13]. Their resource concerns motivate our memory and loading constraints. We study how a changing agent workload reaches those constraints. The resulting bounds can inform capacity planning and adapter retention through expected future demand.

While the spectral threshold and expected total population are classical in multitype branching processes [1], standard models assume stationary, history-independent reproduction. We introduce history-dependent conditional offspring dominating operators to accommodate non-Markovian agent reasoning traces and couple attempt counts directly to grouped scheduling constraints. The scheduling bounds build on Graham's analysis of precedence constraints [2]. The convex design result uses log convexity of nonnegative matrix spectral radii and geometric programming [14, 15]. In our live experiments, DataAgentBench, SWE bench, and ToolSandbox are used as the task validators [6, 7, 4].

3 PRELIMINARIES

3.1 Model and Constraints

A request $r \in \mathcal{R}$ has type $k(r) \in [K]$ and is assigned to one of M workers. Each worker processes ordered, nonempty windows. Requests in the same window start and finish together. Table 1 lists the main parameters. All times and resource costs are nonnegative.

For input length L_r^{in} and predicted output length $\widehat{L}_r^{\mathrm{out}}$, use profiled worker coefficients to set

$$\begin{aligned} P_{rm} &= \alpha_m L_r^{\mathrm{in}} + \beta_m \widehat{L}_r^{\mathrm{out}}, \\ V_{rm} &= \gamma_m (L_r^{\mathrm{in}} + \widehat{L}_r^{\mathrm{out}}). \end{aligned} \tag{1}$$

Equation (1) estimates service time and memory from token lengths.

The binary variables x, g, u obey the following constraints for every relevant request, type, and occupied window.

$$\sum_{m,h} x_{rhm} = 1, \qquad x_{rhm} \le g_{k(r)hm}, \tag{2a}$$

$$\sum_{k} L_{km} g_{khm} \le \Gamma_m, \tag{2b}$$

$$B_m + \sum_{k} A_{km} g_{khm} + \sum_{r} V_{rm} x_{rhm} \le \bar{M}_m, \tag{2c}$$

$$\begin{aligned} &u_{k1m} = g_{k1m}, \\ &u_{khm} = g_{khm}(1 - g_{k,h-1,m}) \quad (h > 1). \end{aligned} \tag{2d}$$

Equation (2a) assigns each request once and activates its type. Equation (2b) limits the types that share a window. Equation (2c) accounts for the base model, active adapters, and request memory. Equation (2d) charges a load when a type becomes active after an inactive window.

Under an additive service model, the duration is

$$d_{hm} = \sum_{r} P_{rm} x_{rhm} + \sum_{k} F_{km} g_{khm} + \sum_{k} S_{km} u_{khm}. \tag{3}$$

Equation (3) combines request work, window activation, and adapter loading.

Table 1: Notations

Symbol	Meaning
r, k, h, m	Request, type, window, and worker
P_{rm}, V_{rm}	Predicted service time and request memory
$B_m, \bar{M}_m$	Base model memory and worker capacity
A_{km}, S_{km}, F_{km}	Adapter memory, load time, activation time
L_{km}, Γ_m	Grouping weight and worker budget
$x_{rhm}, g_{khm}, u_{khm}$	Assignment, active type, and new load
A_r, t_w, e_w, d_w	Release, window start, end, and duration
z_0, H, Q, C	Root counts, offspring, retry, logical child matrices
T, D_{gen}, D_G	Attempts, generating depth, full DAG depth
b_i, K_0, v, δ	Silent error bound, child cap, weights, margin

Let $G = (\mathcal{R}, E)$ be the request DAG. An edge (r, s) requires r to finish before s starts. Write $w(r)$ for the window of r. The timing constraints and objective are

$$\begin{aligned} & e_w = t_w + d_w, \qquad t_{w(r)} \geq A_r, \\ & t_{w(s)} \geq e_{w(r)} \quad ((r,s) \in E), \\ & t_{h+1,m} \geq e_{hm}, \qquad c_r = e_{w(r)}, \qquad (4a) \\ & \min \quad C_{\max}, \qquad C_{\max} \geq c_r \quad (r \in \mathcal{R}). \qquad (4b) \end{aligned}$$

Equation (4a) enforces releases, precedence, and worker order. A_r is an external release time. Completion occurs at the window end. Equation (4b) minimizes the last completion time.

The request DAG models causal precedence, while serving windows enforce worker and memory constraints. Distinguishing them prevents conflating resource driven serialization with data dependencies, enabling accurate adapter reuse accounting. The window graph in Section 4 combines both orders.

3.2 Branching and Validation

Each new call has one generating parent. A retry has its failed attempt as parent. These edges form a forest with deterministic root count row vector z_0 and at least one root. Other prerequisites, including joins, remain in the full DAG. Let T count all attempts. The depths D_{gen} and D_G count edges in the generating forest and full DAG, respectively. Before attempt a of known type i, the history $\mathcal{F}_a$ contains earlier outcomes and scheduling decisions. Its child count vector N_a includes any retry. We assume

$$\mathbb{E}[N_{a,j} \mid \mathcal{F}_a] \leq H_{ij}, \qquad H \geq 0. \qquad (5)$$

Equation (5) bounds the conditional child means entrywise in every admissible history $\mathcal{F}_a$. It permits dependence on earlier outcomes and scheduling decisions. A silent error is an incorrect output accepted by its local validator. Its conditional probability is at most b_i.

For speculative children with mean bound B and at most one retry of the same type, write

$$H = B + Q, \qquad Q = \operatorname{diag}(a_1, \ldots, a_K). \qquad (6)$$

Here a_i bounds the retry probability. In a homogeneous model with correctness probability s_i and detection probability d_i given an error,

$$a_i = (1 - s_i)d_i, \qquad b_i = (1 - s_i)(1 - d_i). \qquad (7)$$

Equation (7) separates detected failures from silent errors. Exact identities below use the stated homogeneous model. Conditional bounds suffice for the upper bounds.

4 THEORETICAL GUARANTEES

We first connect serving windows to completion time, then bound generated work and compare validation policies. Each result has a proof sketch here and a full proof in Appendix A.

4.1 Completion time from the window graph

Lemma 1 (Window graph)**.** Fix a feasible assignment and positive window durations. Let D_π contain the occupied windows, worker order edges, and all edges $w(r) \to w(s)$ induced by E. Timing is feasible exactly when precedence edges cross windows and D_π is acyclic. Its earliest schedule satisfies

$$e_w = d_w + \max\left\{ \max_{r \in w} A_r,\ \max_{v \to w} e_v \right\}. \qquad (8)$$

The predecessor maximum is zero at a source. Thus $C_{\max}$ is a longest weighted path with release sources.

Proof sketch. A timing cycle contradicts positive duration. On an acyclic graph, topological recursion gives the earliest feasible end of every window. □

Proposition 1 (Scheduling lower bound)**.** Under (3), let $p_r^{\min} = \min_m P_{rm}$. Every feasible schedule obeys

$$C_{\max} \geq \max\left\{ \frac{\sum_r p_r^{\min}}{M},\ \max_{\mathcal{P} \in \text{Paths}(G)} \sum_{r \in \mathcal{P}} p_r^{\min} \right\}. \qquad (9)$$

Proof sketch. Total work uses at most $MC_{\max}$ worker time. Requests on a precedence path execute in distinct, ordered windows. □

4.2 Work, error exposure, and runtime tails

Theorem 1 (Expected work and depth)**.** Suppose (5) holds and $\rho(H) < 1$. Let Z_g count attempts by type at generation g. For every nonnegative type cost vector c, we have

$$\mathbb{E}Z_g \le z_0 H^g, \tag{10}$$

$$\mathbb{E}T \le z_0(I-H)^{-1}\mathbf{1}, \tag{11}$$

$$\mathbb{E}\sum_a c_{k(a)} \le z_0(I-H)^{-1}c, \tag{12}$$

$$\mathbb{P}(\text{any silent error}) \le \min\{1, z_0(I-H)^{-1}b\}. \tag{13}$$

For any $\lambda \in (\rho(H), 1)$, there is $v > 0$ with $Hv \le \lambda v$. Writing $v_- = \min_i v_i$ gives

$$\mathbb{P}(D_{\text{gen}} \ge g) \le \frac{z_0 v}{v_-}\lambda^g. \tag{14}$$

Proof sketch. Iterate conditional child means and sum $\sum_g H^g$. Apply a union bound to silent errors and Markov's inequality to the weighted generation count $Z_g v$. □

The row vector $z_0(I-H)^{-1}$ bounds demand by type. We note that while H provides rigorous tail safety certificates, this uniform envelope can be conservative when error rates decay with depth or under absorbing success states. It can inform adapter retention and the storage budget for attempt records. Furthermore, active attempts are bounded by T, yielding conservative reservations for active state under attempt memory caps. Since joins allow D_G to exceed D_{gen}, we use the universal relation $D_G \le T - 1$.

Theorem 2 (Exponential attempt and runtime tails)**.** Assume (5), $\sum_j N_{a,j} \le K_0$ almost surely, and $Hv \le (1-\delta)v$ for $v > 0$, $0 < \delta < 1$. Set $v_+ = \max_i v_i$. For

$$0 < \theta \le \frac{4\delta v_-}{(K_0+1)^2 v_+^2}, \tag{15}$$

every integer $n \ge 0$ satisfies

$$\mathbb{P}(T > n) \le \exp\left(\theta z_0 v - \frac{\theta\delta v_- n}{2}\right). \tag{16}$$

If all releases are at most A_* and occupied windows last at most d_*, then the earliest feasible schedule obeys

$$\mathbb{P}(C_{\max} > A_* + d_* n) \le \exp\left(\theta z_0 v - \frac{\theta\delta v_- n}{2}\right). \tag{17}$$

Also $\mathbb{P}(D_G \ge n) \le \mathbb{P}(T > n)$.

Proof sketch. Exploring an attempt reduces the expected weighted pending queue by at least δv_-. A conditional exponential bound controls its emptying time. Lemma 1 gives $C_{\max} \le A_* + d_* T$. □

Theorem 3 (Tail bound)**.** Under Theorem 2, let $\sigma_v^2 > 0$ bound the conditional variance of $\sum_j v_j N_{a,j}$ in every history. Define

$$I_B = \frac{\delta^2 v_-^2}{2(\sigma_v^2 + \delta v_- K_0 v_+/3)}, \qquad \mu_T = \frac{z_0 v}{\delta v_-}. \tag{18}$$

Then $\mathbb{E}T \le \mu_T$ and

$$\mathbb{P}(T > n) \le e^{-I_B(n-2\mu_T)}. \tag{19}$$

The same bound applies to the runtime event in (17). For one type with offspring $2\,\text{Bern}((1-\delta)/2)$ and its exact variance, $I_B/\Lambda^*(0) \to 1$ as $\delta \downarrow 0$, where $\Lambda^*(0) = -\frac{1}{2}\log(1-\delta^2)$ is the exact tail exponent.

Proof sketch. Use the conditional Bernstein bound for the centered queue increment. Its variance controls the leading exponent. The offset $2\mu_T$ limits the useful range to sufficiently large n. □

4.3 Validation before descendant launch

Proposition 2 (Speculative retry threshold)**.** For $B \ge 0$ and diagonal Q with $a_i < 1$,

$$\rho(B+Q) < 1 \iff \rho((I-Q)^{-1}B) < 1. \tag{20}$$

For an independent multitype process with exact mean $B+Q$, expected total work is infinite if a critical or supercritical class is reachable from a root. An irreducible nonsingular process with spectral radius above one has positive survival probability.

Proof sketch. The inequalities $(B+Q)v < v$ and $(I-Q)^{-1}Bv < v$ are equivalent for $v > 0$. Standard branching results give the statements for the independent process [1]. □

Proposition 3 (Validation before forking)**.** Suppose retries have probability $a_i < 1$ at each type i attempt. Children launch only after a nonretry outcome, with conditional mean row $C_{i\cdot}$. Then

$$\begin{aligned} &H_{\text{gate}} = Q + (I-Q)C, \\ &\rho(H_{\text{gate}}) < 1 \iff \rho(C) < 1. \end{aligned} \tag{21}$$

In the homogeneous model (7), resolving a logical type i request takes $(1-a_i)^{-1}$ expected attempts and has silent acceptance probability $b_i/(1-a_i)$.

Proof sketch. Condition on retry or resolution. The matrix factorization $I - H_{\text{gate}} = (I-Q)(I-C)$ gives the threshold through positive vector inequalities. Sum the geometric retry probabilities for the two scalar identities. □

Proposition 4 (Excess work from speculation)**.** For exact mean operators $H_{\text{spec}} = C + Q$ and $H_{\text{gate}} = Q + (I - Q)C$ with $\rho(H_{\text{spec}}) < 1$,

$$\begin{aligned}\mathbb{E}T_{\text{spec}} - \mathbb{E}T_{\text{gate}} = z_0(I - H_{\text{spec}})^{-1}QC \\ \cdot (I - H_{\text{gate}})^{-1}\mathbf{1} \geq 0.\end{aligned} \tag{22}$$

In one type, the ratio is $1 + aC/(1 - C - a)$.

Proof sketch. Apply the inverse matrix difference identity to $H_{\text{spec}} - H_{\text{gate}} = QC \geq 0$. □

With one type, correctness $s = 0.6$, and speculative child mean 0.8, detection $d = 0.25$ gives 10 expected attempts. At $d = 0.49$, the expectation is 250. Validation before launch with logical mean $C = 0.8$ instead gives $5/(1 - 0.4d)$. Appendix A.6 also derives the distinct effects on silent error counts.

The controlled live protocol gives a finite count comparison. Let b children follow one root and precede one join. Suppose one root perturbation is detected, the root retry succeeds, and all subsequent local outputs pass. Gate checks the root before launch. Speculation executes its direct children before the check. The baseline has $b + 2$ executions, the retry adds one, and speculation also repeats b invalidated children. Hence

$$\begin{aligned}&T_{\text{gate}} = b + 3, \qquad T_{\text{spec}} = 2b + 3, \\ &T_{\text{spec}} - T_{\text{gate}} = b.\end{aligned} \tag{23}$$

This finite protocol illustrates the repeated descendant mechanism in Proposition 4. Figure 1 compares its count prediction with the observed executions.

4.4 Learning bounds from traces

Theorem 4 (Trace certificate)**.** Assume $\mathbb{E}[N_{a,j} \mid \mathcal{F}_a] = h_{ij}$ is constant within each type and $\sum_j N_{a,j} \leq K_0$. For $0 < \eta \leq 1/4$, let $\hat{h}_{ij}(k)$ average the first k observed offspring counts of type i and set

$$\begin{aligned}\varepsilon_k &= K_0\sqrt{\frac{\log(2K^2k(k+1)/\eta)}{2k}}, \\ \hat{H}^+_{ij} &= \min\{K_0, \hat{h}_{ij}(n_i) + \varepsilon_{n_i}\}.\end{aligned} \tag{24}$$

Use $\hat{H}^+_{ij} = K_0$ when $n_i = 0$. With probability at least $1 - \eta$, $|\hat{h}_{ij}(k) - h_{ij}| \leq \varepsilon_k$ simultaneously at all observed sample sizes. On this event $H \leq \hat{H}^+$, so $\rho(\hat{H}^+) < 1$ certifies subcriticality. With fixed type count, child cap, and weight ratio, a margin δ can be certified using order δ^{-2} observations per type up to logarithmic factors. This dependence is necessary even for one type.

Proof sketch. Apply conditional Hoeffding bounds at each type and sample size, then sum the error probabilities. Positive vector comparison gives certification. A two point testing argument gives the lower bound. Explicit constants appear in Appendix A.8. □

If means differ across known state classes, allocate the error budget across classes and take their entrywise maximum. This requires each class to have a constant conditional mean and all admissible states to be covered. Known impossible transitions can be assigned zero upper bounds. Observed averages alone do not establish (5).

An explicit sufficient observation count shows how the margin enters. For $K_0 \geq 1$ and $\kappa = v_+/v_-$, define

$$\begin{aligned}A &= \frac{8K_0^2K^2\kappa^2}{\delta^2}, \\ n_0 &= \max\left\{2A\log\frac{4K^2}{\eta},\ 8A\log(8A)\right\}.\end{aligned} \tag{25}$$

On the simultaneous confidence event, $n_i \geq n_0$ for every type gives $\rho(\hat{H}^+) \leq 1 - \delta/2$. This sufficient count can be large. It protects every coordinate and all observation counts at once. For one type with at most two children, any test with power at least $3/4$ below mean $1 - \delta$ and false certification probability at most $1/4$ above mean one needs at least $1/(2\delta^2)$ observations.

Theorem 5 (Shared difficulty)**.** Consider a retry chain with one root and latent continuation probability $m \in [0, 1]$ shared by the run. Given m, attempts continue independently. Suppose $\mathbb{P}(m > 0) > 0$ and $\mathbb{E}m < 1$. Write $M_k = \mathbb{E}m^k$ and $m^* = \operatorname{ess\,sup} m$. Then

$$\mathbb{P}(T > n) = M_n, \qquad \mathbb{E}T = \mathbb{E}\frac{1}{1-m} \geq \frac{1}{1-\mathbb{E}m}. \tag{26}$$

Equality holds exactly when m is constant. The conditional continuation probability $h_k = M_k/M_{k-1}$ increases to m^*. If $m^* < 1$, then $M_n \leq (m^*)^n$ and $M_n^{1/n} \to m^*$. If $m^* = 1$ and $\mathbb{P}(1 - m \leq x)$ is bounded above and below by positive multiples of x^α near zero, then $M_n \asymp n^{-\alpha}$ and $\mathbb{E}T < \infty$ exactly when $\alpha > 1$.

Proof sketch. Condition on m and sum the survival probabilities. Surviving histories reweight the latent law by m^{k-1}. Moment ratios and the mass near one give the tail rates. □

Uniform m gives $\mathbb{P}(T > n) = 1/(n + 1)$ despite $\mathbb{E}m = 1/2$. Appendix A.9 gives beta laws and a finite sample impossibility result when an unseen mass at $m = 1$ is allowed.

4.5 Serving design and parallel work

Theorem 6 (Convex policy design)**.** Let $H_{ij}(\theta) = \sum_\ell c_{ij\ell} \exp(\langle a_{ij\ell}, \theta\rangle)$ with $c_{ij\ell} \geq 0$ and a fixed zero pattern. For fixed $z_0, w \geq 0$, both $\rho(H(\theta))$ and

$$\Phi(\theta) = z_0(I - H(\theta))^{-1}w \tag{27}$$

are log convex, with zero values treated by continuity. Thus a convex latency proxy plus $\mu\Phi$, $\mu \geq 0$, can be minimized over $\rho(H) \leq 1 - \delta$ as a convex program. If H is irreducible, this constraint is equivalent to the existence of $v > 0$ satisfying $Hv \leq (1-\delta)v$. For a simple Perron eigenvalue and left and right eigenvectors u, v,

$$\begin{aligned} \partial_k \rho &= \frac{u^\top (\partial_k H) v}{u^\top v}, \\ \partial_k \Phi &= \sum_{i,j} m_i (\partial_k H_{ij}) g_j, \end{aligned} \tag{28}$$

where $m^\top = z_0(I - H)^{-1}$ and $g = (I - H)^{-1}w$.

Proof sketch. Matrix powers expand as sums of exponentials. Take the spectral limit and sum the nonnegative series for the inverse. Differentiate the eigenvalue and inverse for (28) [14, 15]. □

Equation (28) assigns a marginal work cost to each policy change. It can balance a chosen latency proxy against added branching. For reducible matrices, the positive vector constraint is sufficient, but equality at the spectral boundary need not make it necessary.

Proposition 5 (Parallel work and depth)**.** Consider singleton windows on identical workers, processing times $p_r \leq p_*$, releases $A_r \leq A_*$, and a nonpreemptive dispatcher that never leaves a worker idle while a request is ready. With total work W and largest precedence path work L,

$$\begin{aligned} C_{\max} &\leq A_* + W/M + (1 - 1/M)L \\ &\leq A_* + p_*\{T/M + (1 - 1/M)(D_G + 1)\}. \end{aligned} \tag{29}$$

If the full DAG is the generating forest and $Hv \leq \lambda v$ with $\lambda < 1$, then for integers $n, m \geq 0$,

$$\begin{aligned} &\mathbb{P}(C_{\max} > A_* + p_*(n/M + m + 1)) \\ &\quad \leq \mathbb{P}(T > n) + \frac{z_0 v}{v_-}\lambda^{m+1}. \end{aligned} \tag{30}$$

Proof sketch. After the last release, idle worker time is charged to a precedence chain [2]. Use $W \leq p_* T$, $L \leq p_*(D_G + 1)$, and a union bound. □

More workers reduce the work term. They do not remove the dependency term. Grouped windows instead use Lemma 1 and the duration bound in Theorem 2.

4.6 Using the bounds for resource planning

The bounds give explicit planning quantities once a policy has a valid conditional matrix. Let s_* be an upper bound on the bytes stored for each active attempt. For a target overflow probability $0 < \epsilon < 1$, choose

$$n_\epsilon = \left\lceil \frac{2z_0 v}{\delta v_-} + \frac{2\log(1/\epsilon)}{\theta \delta v_-} \right\rceil, \tag{31}$$

where θ satisfies (15). Substituting into (16) gives $\mathbb{P}(T > n_\epsilon) \leq \epsilon$. Since the number of active attempts never exceeds T, a reservation of $s_* n_\epsilon$ bytes covers their state with probability at least $1 - \epsilon$. Base model and adapter memory are added separately through (2c). Under the duration and release assumptions of Theorem 2, the same event also gives

$$C_{\max} \leq A_* + d_* n_\epsilon. \tag{32}$$

These statements follow from the attempt bound and hold together on the event $T \leq n_\epsilon$.

The expected demand by type gives another useful quantity. Define

$$q = z_0(I - H)^{-1}. \tag{33}$$

Then q_i bounds the expected number of type i attempts. A scheduler can compare this demand with the memory and loading cost of the corresponding adapter. Retaining a frequently needed adapter may save repeated loads, while its memory still has to satisfy (2c). Equation (33) supplies demand estimates for that choice. It does not determine an optimal cache policy by itself.

The same matrix also weights error exposure. A policy with $qb \leq \varepsilon$ meets the silent error bound in (13). Increasing a branching rate raises both future work and the opportunities for silent errors when other terms stay fixed. Near the spectral threshold, the inverse matrix becomes sensitive to small changes. This gives a reason to retain a margin below one when selecting speculation or retry budgets. Appendix B gives the exact matrix difference formula and a general serving objective.

5 EXPERIMENTS

We evaluate the theory with CPU simulations, controlled live episodes, natural task runs, and client timing replay. Each setting tests a different part of the analysis.

5.1 CPU checks

For the binary offspring law in Theorem 3, an exact dynamic program gives survival probabilities through

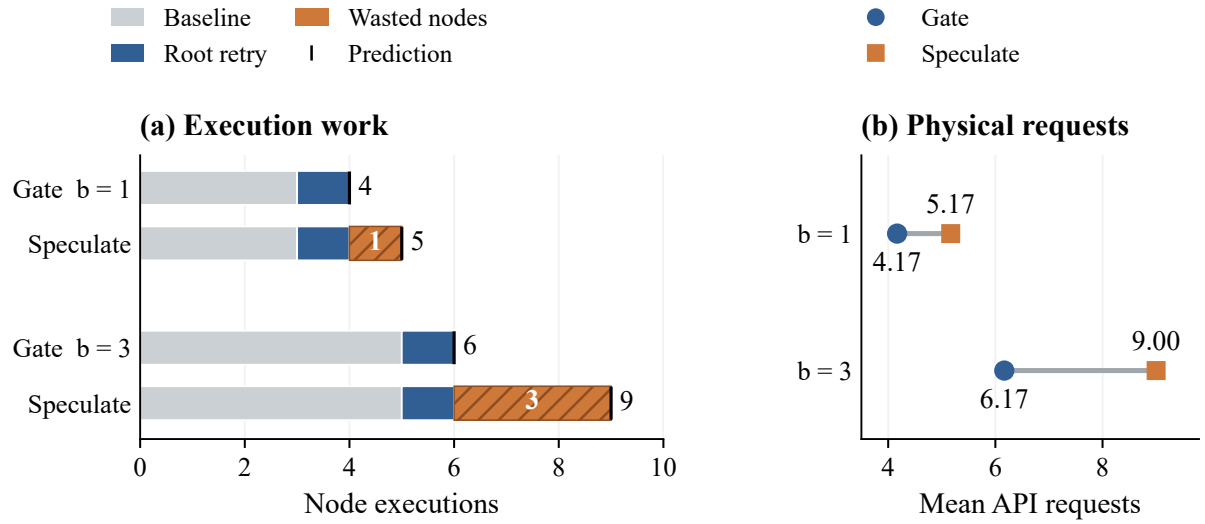


Figure 1: Validation order under one injected root fault. Gate avoids all invalidated descendants and reduces node work by 20% and 33.3% for one and three branches. Physical API requests include transport retries.

$n = 600$. Figure 2 compares the exact curve with the Bernstein and range bounds. The Bernstein exponent ratio rises from 0.829 at $\delta = 0.4$ to 0.987 at $\delta = 0.02$. The range bound ratio changes from 0.408 to 0.444. The Bernstein bound becomes close to the exact rate near the threshold.

The right panel checks the shared difficulty model in Theorem 5. The simulated tails follow the exact beta curves over the sampled range. Uniform difficulty gives $\Pr(T > n) = 1/(n+1)$ even though the mean continuation probability is $1/2$. For Beta$(2, 3)$, the exact expected work is 2.0. Replacing the shared difficulty by its mean gives 1.667. The difference comes from dependence within a run.

Table 2: Trace certification for the bounded two type process. The error budget is $\eta = 0.1$.

Spectral radius	Margin δ	Observations per type
0.60	0.40	1,000
0.80	0.20	5,000
0.90	0.10	50,000
0.95	0.05	100,000

The certification experiment uses $H_\omega = \left(\begin{smallmatrix} 0 & 2\omega \\ \omega/2 & 0 \end{smallmatrix}\right)$ with spectral radius ω. Known zero entries stay fixed. Confidence bounds are applied to the two possible transitions. Table 2 shows that smaller spectral margins need more observations. A separate coverage check at $\omega = 0.8$ records zero violations in 2,000 repetitions. The tested observation counts extend through 3,000.

The parallel scheduling check uses 2,000 generated trees at each of three spectral settings and six worker counts. All 36,000 complete schedules satisfy Proposition 5. More workers reduce the work term. The dependency term remains. The fixed four node example also raises completion from 0.5224 to 0.9867 with at most two retries per node.

We also compare three scheduling policies on common feasible inputs. On 720 CPU instances, the mean makespans are 133.71 for FIFO, 128.50 for adapter affinity, and 129.08 for grouping. On 72 client timing replay instances, FIFO and affinity both average 37.79 seconds. Grouping averages 41.68 seconds. A separate check on 20 small instances gives mean gaps to the optimum of 7.57%, 7.59%, and 19.84%. The replay uses observed client durations and normalized resource costs.

5.2 Validation order

The controlled live check uses three short templates. Each root has one or three direct children. We use two repeated units for each template and branch count. Each unit runs gate and speculation with an unmodified root and with one root perturbation. This gives 12 paired units and 48 logical episodes. Gate validates the root before launching its children. Speculation runs the direct child layer before the root check. Local validators detect every incorrect root output. A rejected root is retried. Figure 1 shows the work difference. The differences are exactly the one and three invalidated children predicted by (23). The comparison therefore isolates extra work under the same final checks. In the root fault conditions, mean physical

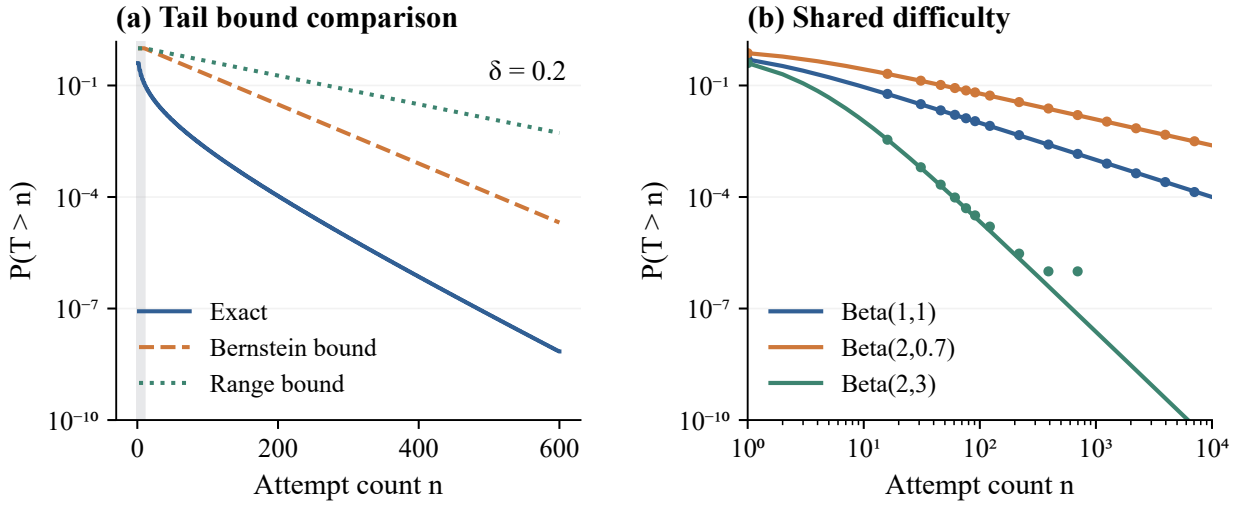


Figure 2: CPU checks of tail predictions. The left panel shows the exact binary offspring survival probability and two bounds at margin $\delta = 0.2$. The Bernstein bound is tighter than the range bound. Shading marks $n < 2\mu_T$. The right panel shows exact survival probabilities for shared beta difficulty and saved Monte Carlo estimates.

API requests are 4.17 for gate and 5.17 for speculation with one branch. They are 6.17 and 9.00 with three branches. These counts include transport retries. A separate CPU scan varies detection strength with correctness probability 0.6 and child mean 0.8. The speculative mean is $0.8 + 0.4d$. It crosses one at $d = 0.5$. The gated mean is $0.4d + (1 - 0.4d)0.8$ and stays below one. The observed capped work follows this split.

5.3 Task recovery

We compare a single attempt, fresh restart, and stateful repair on 18 DataAgentBench tasks [6]. Each task shares its initial execution across the three policies. After rejection, restart begins a fresh execution. Repair keeps the dialogue and execution state. Both policies receive the same generic rejection signal and the same added phase caps. Reference answers stay outside the agent context.

Table 3: Recovery on 18 DataAgentBench tasks. Requests and seconds are means per task. Each policy includes the shared initial execution.

Policy	Complete	Rescues	Requests	Seconds
Single attempt	8/18	0	21.8	264.3
Fresh restart	11/18	3	35.2	416.1
Stateful repair	10/18	2	24.8	346.9

Table 3 shows three rescues from restart and two from repair. Restart raises completion by 16.7 percentage points over the shared initial result. Repair raises it by 11.1 points. Both gains require more requests and more time. The comparison uses the full selected sample. The model is `qwen3.8-27b` with temperature 0.2 and top $p = 1$. Each call allows at most 2,048 output tokens. Each phase permits 100 iterations and 900 seconds. One client channel spaces dispatches by at least 6.5 seconds. These results describe the selected tasks under matched added budgets. A separate 36 task software and tool cohort uses different validators. We keep this cohort separate from the DataAgentBench comparison.

6 CONCLUSION

The offspring matrix links dynamic agent work to error exposure and runtime tails. The window graph links realized work to feasible serving schedules. CPU checks support the tail, certification, and scheduling results. Controlled live episodes show that validation before launch avoids repeated downstream work. Natural data tasks show that recovery can add accepted outcomes at extra cost. These results support bounded recovery and explicit spectral margins when planning agent workloads. Future work should extend the branching process framework to settings where the offspring distribution itself adapts over time, such as workflows that learn error rates from prior runs and adjust retry budgets accordingly. Tightening the gap between the resolvent upper bound and observed expected attempts, particularly under correlated or depth-decaying error models, remains an open and practically important direction.

AI USE STATEMENT

Generative AI tools were utilized to assist with manuscript editing and code implementation. The theoretical results and proofs were first developed and written by the authors, and were subsequently double-checked with the assistance of AI. Full responsibility for the correctness and content of the paper is assumed by the authors.

References

[1] K. B. Athreya and P. E. Ney. Branching Processes. Springer, 1972.

[2] R. L. Graham. Bounds on multiprocessing timing anomalies. SIAM Journal on Applied Mathematics, 17(2), 416–429, 1969.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations, 2023.

[4] J. Lu et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682, 2024.

[5] O. Paharskyi and H. Haina. AgentTrace: Tool-Using Model Telemetry Dataset. Dataset and code repository, 2026. `https://github.com/pagarsky/agent-trace`.

[6] R. Ma et al. Can AI Agents Answer Your Data Questions? A Benchmark for Data Agents. arXiv:2603.20576, 2026.

[7] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? International Conference on Learning Representations, 2024.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. Advances in Neural Information Processing Systems, 36, 8634–8652, 2023.

[9] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. Advances in Neural Information Processing Systems, 36, 46534–46594, 2023.

[10] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. Advances in Neural Information Processing Systems, 36, 11809–11822, 2023.

[11] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami. An LLM Compiler for Parallel Function Calling. Proceedings of the 41st International Conference on Machine Learning, PMLR 235, 24370–24391, 2024.

[12] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. Proceedings of the 29th Symposium on Operating Systems Principles, 611–626, 2023.

[13] Y. Sheng et al. SLoRA: Scalable Serving of Thousands of LoRA Adapters. Proceedings of Machine Learning and Systems, 6, 296–311, 2024.

[14] J. F. C. Kingman. A Convexity Property of Positive Matrices. The Quarterly Journal of Mathematics, 12(1), 283–284, 1961.

[15] S. Boyd, S.-J. Kim, L. Vandenberghe, and A. Hassibi. A Tutorial on Geometric Programming. Optimization and Engineering, 8(1), 67–127, 2007.

CHECKLIST

1. For all models and algorithms presented, check if you include:
   (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes] .
   (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes] .
   (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [Yes].

2. For any theoretical claim, check if you include:
   (a) Statements of the full set of assumptions of all theoretical results. [Yes] .
   (b) Complete proofs of all theoretical results. [Yes].
   (c) Clear explanations of any assumptions. [Yes] .

3. For all figures and tables that present empirical results, check if you include:
   (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [Yes]
   (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Not Applicable].

(c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes] .

(d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [Yes].

4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:

   (a) Citations of the creator If your work uses existing assets. [Yes]

   (b) The license information of the assets, if applicable. [No] .

   (c) New assets either in the supplemental material or as a URL, if applicable. [Yes].

   (d) Information about consent from data providers/curators. [Not Applicable] .

   (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Yes].

5. If you used crowdsourcing or conducted research with human subjects, check if you include:

   (a) The full text of instructions given to participants and screenshots. [Not Applicable].

   (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable] .

   (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable] .

A PROOFS AND FORMULATION DETAILS

A.1 Proof of Lemma 1

Proof. If a precedence edge lies within one window, its successor must start after the common window end while also starting at the common window start. Positive duration rules this out. Along every edge of D_π, the successor window starts after the predecessor ends. A directed cycle would then imply that a window starts strictly after its own start. Thus temporal feasibility implies acyclicity and distinct windows at every precedence edge.

Conversely, suppose the conditions hold. Process D_π in a topological order and define each end time by (8). Each start is its end less its positive duration. The release, precedence, and within worker order inequalities in (4a) all hold. An induction on the same order shows that any other feasible schedule ends each window at least as late, so the recursion is the earliest schedule. Repeated substitution of (8) gives the longest path interpretation. Each request completes at the end of its assigned window by the grouped service convention.

For a mixed integer encoding of $u_{khm} = g_{khm}(1 - g_{k,h-1,m})$, use $u_{khm} \geq g_{khm} - g_{k,h-1,m}$, $u_{khm} \leq g_{khm}$, and $u_{khm} \leq 1 - g_{k,h-1,m}$ for later occupied windows. The first occupied window uses $u = g$. Worker timing uses $t_{h+1,m} \geq e_{hm}$, since (3) already charges the switching time. An assigned request has $c_r = e_{w(r)}$. With indicator constraints this equality and each precedence inequality are immediate. A big U encoding needs a finite, valid upper bound on schedule times. These conditions also prevent empty windows from creating artificial makespan or load costs. □

A.2 Proof of Proposition 1

Proof. Each worker processes nonoverlapping windows of positive duration. The sum of its request processing loads is at most the sum of its window durations, which is at most $C_{\max}$. Summing over M workers shows $\sum_r p_r^{\min} \leq MC_{\max}$. Along a path in G, each request finishes before its successor starts. These requests occupy distinct windows, whose serial durations sum to at most $C_{\max}$ and each dominate the corresponding $p_r^{\min}$. Maximizing over paths gives the second term of (9). □

A.3 Proof of Theorem 1

Proof. Condition on the complete history before each generation g attempt. Summing (5) over these attempts and then taking expectations gives $\mathbb{E}Z_{g+1} \leq (\mathbb{E}Z_g)H$. Induction yields (10). Monotone convergence and the Neumann series $\sum_{g\geq 0} H^g = (I - H)^{-1}$ give (11) and (12). Finite expected total attempts implies finite attempts almost surely. For an attempt a, let I_a indicate a silent error. Conditional on its pre disclosure history, $\mathbb{E}I_a \leq b_{k(a)}$. Sum over the finite generations, apply monotone convergence, and use the union bound. This gives (13) without independence among silent errors.

For any $\lambda \in (\rho(H), 1)$, define $v = (I - H/\lambda)^{-1}\mathbf{1}$. The inverse is a convergent nonnegative series and $v > 0$. Its defining equation gives $Hv = \lambda(v - \mathbf{1}) \leq \lambda\mathbf{v}$. From (10), $\mathbb{E}(Z_g v) \leq z_0 H^g v \leq \lambda^g z_0 v$. On $D_{\text{gen}} \geq g$ there is at least one type present at generation g, so $Z_g v \geq \min_i v_i$. Markov's inequality gives (14).

For (35), condition generation by generation to obtain $\mathbb{E}(Z_{g+1}v) \leq \lambda_g \mathbb{E}(Z_g v)$. Iteration and the same Markov argument give the displayed product. A path in a finite DAG visits distinct attempts. Thus $D_G \leq T - 1$. □

A.4 Proof of Theorem 2

Proof. Explore the generating forest in the parent before child disclosure order specified in Section 3. The history at each exploration contains all earlier disclosed marks and scheduling decisions. Let Q_n be the sum of the v weights of discovered but unexplored attempts after n explorations, and let τ be the first time the queue becomes empty. The exploration visits all attempts, so $\tau = T$. While $Q_n > 0$, exploring a type i vertex changes its weight by $\Delta = X - v_i$, where X is the total v weight of its newly discovered children. The child cap gives $0 \leq X \leq K_0 v_+$. Conditional on the past, $\mathbb{E}\Delta \leq -\delta v_i \leq -\delta v_-$. The range of Δ has width at most $(K_0 + 1)v_+$. Hoeffding's conditional exponential bound therefore yields

$$\mathbb{E}[e^{\theta\Delta} \mid \mathcal{F}_n] \leq \exp\left(-\theta\delta v_- + \frac{\theta^2(K_0+1)^2 v_+^2}{8}\right) \leq e^{-\theta\delta v_-/2}$$

for (15). Induction over active queue states gives

$$\mathbb{E}\big[e^{\theta Q_n}\mathbf{1}\{T > n\}\big] \le e^{\theta z_0 v - \theta\delta v_- n/2}.$$

Since $e^{\theta Q_n} \ge 1$ on $T > n$, this proves (16). The queue inequality remains valid when offspring of different vertices are dependent, since only their conditional means and a per vertex cap were used.

There are at most T occupied serving windows. In the window graph, every directed path visits at most that many distinct windows. Each exogenous release is at most A_* and each window weight is at most d_*. Lemma 1 therefore gives $C_{\max} \le A_* + d_* T$. The event in (17) implies $T > n$ for integer n. The DAG path bound follows from $D_G \le T - 1$. □

A.5 Proof of Proposition 2

Proof. Let $D = I - Q$, whose diagonal entries are positive. Then

$$I - (B + Q) = D - B = D\big[I - D^{-1}B\big].$$

For a nonnegative square matrix A, $\rho(A) < 1$ is equivalent to existence of a strictly positive vector x with $Ax < x$. If $B+Q$ has such a vector, $(B+Q)x < x$ rearranges to $D^{-1}Bx < x$. Conversely $D^{-1}Bx < x$ rearranges to $(B + Q)x < x$. This proves (20), including reducible matrices. For the independent process, the expected generation counts equal $z_0 H^g$. If a reachable communicating class has spectral radius at least one, the series of expected counts diverges. In the irreducible nonsingular case with spectral radius greater than one, the standard multitype branching survival argument gives positive survival probability [1]. □

A.6 Proof of Proposition 3

Proof. At a type i attempt, a retry is created with probability a_i. With the remaining probability $1 - a_i$, the logical request resolves and then generates children according to row $C_{i\cdot}$. Taking conditional means gives (21). Its subcriticality is equivalent to $\rho(C) < 1$ by the same positive vector argument as in the preceding proof, since $I - H_{\text{gate}} = (I - Q)(I - C)$. The attempts before resolution form a geometric random variable with continuation probability a_i, giving mean $(1-a_i)^{-1}$. A silent acceptance on the $(j+1)$st attempt has probability $a_i^j b_i$. Summing over $j \ge 0$ gives $b_i/(1 - a_i)$. Independent attempts are sufficient for the geometric identities. For the single type speculative model, the progeny mean is $\beta + a$ and the expected number of attempts is $(1 - \beta - a)^{-1}$. Multiplying by the silent error probability b gives (37). Differentiation gives

$$\frac{d}{dd}\mathbb{E}N_{\text{silent}} = \frac{(1-s)(\beta - s)}{[1 - \beta - (1-s)d]^2}.$$

Under a gate, each logical vertex yields $b/(1-a)$ expected silent acceptances and the expected number of logical vertices is $(1 - C)^{-1}$. Its derivative in d is $-s(1-s)/[(1-C)(1-(1-s)d)^2]$.

□

A.7 Auxiliary probability and matrix lemmas

Lemma 2 (Positive matrix comparison facts)**.** For $A \ge 0$. (i) $Av \le sv$, $v > 0$, implies $\rho(A) \le s$. (ii) if $\rho(A) < s$, there is $v > 0$ with $Av < sv$. (iii) $0 \le A \le A'$ and $\rho(A') < 1$ imply $(I - A)^{-1} \le (I - A')^{-1}$. And (iv), if $Av \le \lambda v$ with $\lambda < 1$, then $(I - A)^{-1}\mathbf{1} \le v/((1-\lambda)v_-)$.

Proof. For (i), $A^k\mathbf{1} \le A^k v/v_-$ and Gelfand's formula give the claim. For (ii), take $v = (I - A/s')^{-1}\mathbf{1}$ for $s' \in (\rho(A), s)$. For (iii), compare the Neumann series term by term. For (iv), sum $A^g v \le \lambda^g v$ and use $\mathbf{1} \le v/v_-$. □

Lemma 3 (Conditional Bernstein mgf)**.** If $|Y| \le b$, $\mathbb{E}[Y \mid \mathcal{F}] = 0$, and $\mathbb{E}[Y^2 \mid \mathcal{F}] \le \sigma^2$, then for $0 < \theta < 3/b$,

$$\mathbb{E}[e^{\theta Y} \mid \mathcal{F}] \le \exp\left(\frac{\theta^2\sigma^2}{2(1 - \theta b/3)}\right).$$

Proof. Expand $e^u - 1 - u$ and use $k! \ge 2\,3^{k-2}$ for $k \ge 2$. The centered linear term vanishes and the resulting geometric series gives the display. □

Lemma 4 (Weighted exploration queue)**.** Explore attempts parent before child and let Q_n be the total v weight of discovered but unexplored attempts. Then $Q_0 = z_0 v$, $T = \min\{n : Q_n = 0\}$, and an attempt of type i changes the queue by $\Delta = X - v_i$ with $X = \sum_j v_j N_{a,j} \in [0, K_0 v_+]$. If $Hv \le \lambda v$, then $\mathbb{E}[X \mid \mathcal{F}_a] \le \lambda v_i$.

Proof. This follows directly from the queue definition and summing $v_j \mathbb{E}[N_{a,j} \mid \mathcal{F}_a] \le v_j H_{ij}$. □

A.8 Proof of Theorem 4

Proof. For a fixed type and child coordinate, observe offspring only when that type is selected. Selection occurs before its mark is disclosed. The centered values are therefore martingale differences with range length K_0. Conditional Hoeffding bounds give, on observing at least k values,

$$\mathbb{P}\{|\hat{h}_{ij}(k) - h_{ij}| > \varepsilon_k,\ n_i \ge k\} \le \frac{\eta}{K^2 k(k+1)}.$$

One can obtain this inequality from the exponential supermartingale stopped at the kth type observation. Localization at a finite global exploration time and Fatou's lemma also cover a type that stops appearing. No artificial zero offspring observations are added. Summing over i, j, k proves the simultaneous event. On this event $H \le \hat{H}^+$. Monotonicity of matrix powers and their nonnegative sums proves that the upper matrix gives valid work bounds. Positive vector comparison proves the spectral claim.

For explicit sufficient constants, suppose $Hv \le (1-\delta)v$, put $\kappa = v_+/v_-$, and assume $K_0 \ge 1$. Define

$$A = \frac{8K_0^2 K^2 \kappa^2}{\delta^2}, \qquad n_0 = \max\left\{2A \log \frac{4K^2}{\eta},\ 8A \log(8A)\right\}. \tag{34}$$

If every $n_i \ge n_0$, then $2\varepsilon_{n_i} K\kappa \le \delta/2$. To check this, it suffices to show $n \ge A \log(2K^2 n(n+1)/\eta)$. For $n \ge 1$, the logarithm is at most $\log(4K^2/\eta) + 2\log n$. The first term is at most $n/(2A)$ by the first condition. The second is at most $n/(2A)$ when $n \ge 8A\log(8A)$, using that $\log n/n$ decreases for $n > e$. Hence

$$(\hat{H}^+ v)_i \le (1 - \delta + 2\varepsilon_{n_i} K\kappa) v_i \le (1 - \delta/2) v_i.$$

If $K_0 = 0$, there are no children and certification is immediate.

For the lower bound, compare the one type laws $2\,\mathrm{Bern}(1/2)$ and $2\,\mathrm{Bern}((1-\delta)/2)$, with $0 < \delta < 1$. A test with power at least $3/4$ at the second law and false certification probability at most $1/4$ at the first has a probability gap at least $1/2$. Their one observation relative entropy is at most their chi squared divergence δ^2. Pinsker's inequality bounds the n observation total variation by $\sqrt{n\delta^2/2}$. Thus $n \ge 1/(2\delta^2)$. □

A.9 Proof of Theorem 5

Proof. Given m, surviving beyond n attempts has probability m^n. Taking expectations gives M_n. Summing these nonnegative probabilities gives $\mathbb{E}T = \mathbb{E}[1/(1-m)]$. Strict convexity of $x \mapsto 1/(1-x)$ proves the inequality and equality condition because $\mathbb{E}m < 1$. If $m = 0$ almost surely, the omitted trivial case has $T = 1$.

Given survival through attempt $k-1$, the latent law is reweighted by m^{k-1}. Thus the next continuation probability is $h_k = M_k/M_{k-1}$. Cauchy Schwarz gives $M_k^2 \le M_{k-1} M_{k+1}$, so h_k is nondecreasing. For each $c < m^*$ with $c > 0$, $M_n \ge c^n \mathbb{P}(m \ge c)$, while $M_n \le (m^*)^n$. Taking roots and then $c \uparrow m^*$ shows $M_n^{1/n} \to m^*$. Since $M_n = \prod_{k=1}^n h_k$, the monotone ratios have the same limit.

For the polynomial case, write $X = 1 - m$ and let $F(x) = \mathbb{P}(X \le x)$. For all large n, the event $X \le 1/n$ gives the lower bound $\mathbb{E}(1-X)^n \ge cn^{-\alpha}$ for some $c > 0$. For the upper bound, use $(1-X)^n \le e^{-nX}$ and integration by parts to obtain

$$\mathbb{E}e^{-nX} = e^{-n} + n\int_0^1 e^{-nx} F(x)\, dx.$$

The part near zero is at most a constant times $n^{-\alpha}$, using $F(x) \le c_2 x^\alpha$. The remaining part is exponentially small. Thus $M_n \asymp n^{-\alpha}$. Summing M_n gives the criterion $\alpha > 1$. □

For $m \sim \mathrm{Beta}(a, b)$ with $a, b > 0$, beta integration yields

$$\mathbb{P}(T > n) = \frac{B(a+n, b)}{B(a, b)} \sim \frac{\Gamma(a+b)}{\Gamma(a)} n^{-b}.$$

For $b > 1$, the exact expected work is $(a + b - 1)/(b - 1)$. Replacing m by its mean instead gives $(a + b)/b$. Uniform difficulty has $M_n = 1/(n + 1)$ and infinite expected work.

There is also a finite sample limit when the model class permits an unseen mass at $m = 1$. Let μ_0 be a latent law supported on $[0, 1/2]$ and set $\mu_1 = (1-\epsilon)\mu_0 + \epsilon\delta_1$. Couple each of n independent runs so that its latent value agrees under both laws unless the contaminating component is drawn. The probability of any disagreement is at most $n\epsilon$. Any certification event with probability at most α under μ_1 therefore has probability at most $\alpha + n\epsilon$ under μ_0. The law μ_1 has infinite expected work. Since ϵ can be arbitrarily small, no procedure can have useful power uniformly over this unrestricted class while controlling false certification.

A.10 Proof of Theorem 3

Proof. The weighted resolvent lemma gives $\mathbb{E}T \le \mu_T$. In the exploration queue write $\Delta = Y + (\mathbb{E}[X \mid \mathcal{F}_a] - v_i)$, where Y is centered, $Y \le b = K_0 v_+$ and $\mathbb{E}[Y^2 \mid \mathcal{F}_a] \le \sigma_v^2$. The conditional Bernstein lemma with

$$\theta^* = \frac{\delta v_-}{\sigma_v^2 + \delta v_- b/3}$$

gives a conditional factor at most $\exp(-\theta^* \delta v_-/2)$. Induction over active queue states yields

$$\mathbb{E}[e^{\theta^* Q_n} \mathbf{1}\{T > n\}] \le e^{\theta^* z_0 v - n\theta^* \delta v_-/2}.$$

Since $Q_n > 0$ on $\{T > n\}$, the tail bound follows with $I_B = \theta^* \delta v_-/2$. The makespan event implies $T > n$ because an earliest window schedule has at most T nonempty windows of duration d_*.

For the exponent comparison, put $p = (1 - \delta)/2$. A binary tree with k internal vertices has $2k + 1$ attempts. Its probability is $C_k p^k (1 - p)^{k+1}$, where $C_k = (k + 1)^{-1} \binom{2k}{k}$. Stirling's formula gives the exponential rate $-\frac{1}{2} \log(4p(1 - p)) = -\frac{1}{2} \log(1 - \delta^2)$. The exact offspring variance is $1 - \delta^2$, so

$$I_B = \frac{\delta^2}{2(1 - \delta^2 + 2\delta/3)}.$$

Both exponents equal $\delta^2/2 + o(\delta^2)$ as $\delta \downarrow 0$. This proves the limiting ratio.

□

A.11 Proof of Theorem 6

Proof. Each product entry of $H(\theta)^k$ is a sum of exponentials of affine functions. Thus $s_k(\theta) = \mathbf{1}^\top H(\theta)^k \mathbf{1}$ is log convex. If the fixed graph contains a cycle, Gelfand's formula gives $k^{-1} \log s_k \to \log \rho(H)$. Pointwise limits preserve the convex inequality. If the graph is acyclic, H is nilpotent and its spectral radius is identically zero. This also gives log convexity with the usual zero convention. This is a form of the classical spectral log convexity result [14].

Likewise, each partial sum $\sum_{g=0}^{n} z_0 H^g w$ is a nonnegative sum of exponentials. Its log convex inequality passes to the increasing limit on $\rho(H) < 1$. This proves the claim for Φ. A nonnegative log convex function is convex. The sublevel set $\rho(H) \le 1 - \delta$ is convex and lies in the convergence domain, so the stated objective and feasible set form a convex program.

For any nonnegative matrix, $Hv \le sv$ with $v > 0$ implies $\rho(H) \le s$. If H is irreducible, its positive Perron vector also gives the converse at $s = 1 - \delta$. With $x = \exp \theta$, dividing each row inequality by sv_i gives the geometric programming form [15]. Without irreducibility, equality at the boundary can fail to admit a positive vector. For example, $H = \left(\begin{smallmatrix} s & 1 \\ 0 & s \end{smallmatrix}\right)$ has radius s but no $v > 0$ with $Hv \le sv$. A strict inequality $\rho(H) < s$ always admits such a vector.

At a simple Perron eigenvalue, differentiate $Hv = \rho v$ and left multiply by $u^\top$. The terms containing $\partial_k v$ cancel, giving the first identity in (28). Differentiating $(I - H)^{-1}$ gives the second identity. These gradients enter the usual Karush Kuhn Tucker conditions when a constraint qualification such as Slater's condition holds. □

A.12 Proof of Proposition 4

Proof. Since $H_{\mathrm{spec}} - H_{\mathrm{gate}} = QC \geq 0$, the comparison lemma orders the spectral radii and the resolvents. Applying the resolvent identity gives

$$(I - H_{\mathrm{spec}})^{-1} - (I - H_{\mathrm{gate}})^{-1} = (I - H_{\mathrm{spec}})^{-1} QC (I - H_{\mathrm{gate}})^{-1},$$

which proves the matrix expression. In one type, substitute the scalar resolvents $1/(1-C-a)$ and $1/((1-a)(1-C))$. □

A.13 Proof of Proposition 5

Proof. Assume $C_{\max} > A_*$. Choose a job finishing at $C_{\max}$ and recursively choose a latest finishing predecessor, obtaining a precedence chain. At any time after A_* when a worker is idle, the first unfinished job on this chain is released and has all predecessors complete. Work conservation therefore forces it to be executing. Idle time has measure at most L, so idle worker area is at most $(M-1)L$. Counting busy and idle area gives $M(C_{\max} - A_*) \leq W + (M-1)L$. The second inequality uses $W \leq p_* T$ and $L \leq p_*(D_G + 1)$. If $T \leq n$ and $D_G \leq m$, this proves the displayed tail event. A union bound with the generation depth bound gives the last assertion. □

A.14 Changing offspring bounds

For generation dependent bounds H_g, suppose one vector $v > 0$ satisfies $H_g v \leq \lambda_g v$ for every g. Iterating conditional weighted counts gives

$$\mathbb{P}(D_{\mathrm{gen}} \geq g) \leq \frac{z_0 v}{v_-} \prod_{\ell=0}^{g-1} \lambda_\ell. \tag{35}$$

The condition concerns a common vector. Individual spectral radii do not control a product. For example,

$$H_{2t} = \begin{pmatrix} 0 & 2 \\ 0 & 0 \end{pmatrix}, \qquad H_{2t+1} = \begin{pmatrix} 0 & 0 \\ 2 & 0 \end{pmatrix}. \tag{36}$$

Each matrix has spectral radius zero. Their product is $\mathrm{diag}(4, 0)$. Deterministic offspring with these alternating matrices produce 4^t type one requests after $2t$ generations from one type one root.

Joins give a separate distinction between depths. A precedence chain of n root requests has generating depth zero and full DAG depth $n-1$. Every DAG path visits distinct attempts, so $D_G \leq T - 1$ still holds.

A.15 Silent error counts under the two validation orders

In one type, let $\beta < 1$ be the speculative child mean and suppose $\beta + (1-s)d < 1$. The exact mean identities give

$$\mathbb{E}T = \frac{1}{1 - \beta - (1-s)d}, \qquad \mathbb{E}N_{\mathrm{silent}} = \frac{(1-s)(1-d)}{1 - \beta - (1-s)d}. \tag{37}$$

As shown in the proof of Proposition 3, $\frac{d}{dd}\mathbb{E}N_{\mathrm{silent}}$ has the sign of $\beta - s$. Stronger detection can therefore increase the expected number of silent incidents when speculative work grows fast enough. For validation before launch,

$$\mathbb{E}N_{\mathrm{silent}} = \frac{b}{(1-a)(1-C)}.$$

This expression decreases with d when $s > 0$ and $C < 1$. Expected incident counts differ from the probability of at least one incident. The experiment reports both quantities separately.

B SERVING SCOPE

A serving policy can change both workload and schedule. Prompts, retained context, timeouts, and validation can change H_π. Grouping changes feasible windows and their durations. If grouping only places a fixed request set, it leaves H_π unchanged.

A risk constrained policy may use

$$\begin{aligned} \min_{\pi} \quad & \mathbb{E}C_{\max}(\pi) \\ \text{subject to} \quad & z_0(I - H_\pi)^{-1} b_\pi \le \varepsilon, \\ & \rho(H_\pi) < 1, \\ & \text{constraints (2) and (4).} \end{aligned} \tag{38}$$

This problem need not be convex. The convex result in the main text applies only to its stated parameterization.

For two subcritical nonnegative matrices,

$$(I - H')^{-1} - (I - H)^{-1} = (I - H')^{-1}(H' - H)(I - H)^{-1}. \tag{39}$$

An entrywise increase in the offspring bound increases the resolvent work bound. If each active attempt stores at most s_* bytes, active state is at most s_*T bytes. The attempt tail bound then gives a conservative state reservation after fixed model and adapter memory are added.

The scheduling layer applies to a realized finite request set. The offspring bound is a condition on the agent and policy across admissible histories. Trace certification therefore needs fixed state classes and coverage of every admissible class. A held out audit can reveal missing states. It cannot certify states that were not covered.

C EXPERIMENTAL DETAILS

The evidence sources stay separate. CPU simulations test model predictions. Controlled live episodes test finite execution behavior. Natural tasks test recovery cost. Client replay tests schedules using observed durations.

C.1 CPU checks

The CPU checks use seed 20261005. The certification study uses 2,000 repetitions for each (ω, n) cell with $\eta = 0.1$ and

$$\omega \in \{0.6, 0.8, 0.9, 0.95\}.$$

The tested sample sizes are

$$n \in \{50, 100, 200, 500, 1000, 2000, 5000, 10^4, 5{\cdot}10^4, 10^5, 2{\cdot}10^5\}.$$

Known zero transitions remain fixed at zero.

The shared difficulty study uses one million runs for each of Beta$(1, 1)$, Beta$(2, 0.7)$, and Beta$(2, 3)$. The attempt cap is 10^5. Capped runs are treated as censored. The Bernstein comparison uses the exact one type dynamic program through $n = 600$. The tested margins are 0.4, 0.2, 0.1, 0.05, and 0.02.

The convex design check uses a three type posynomial model with $\rho(H) \le 0.85$. It uses 3,000 random midpoint checks. The work and depth check uses 2,000 trees at each $\omega \in \{0.6, 0.8, 0.9\}$ and worker counts $M \in \{1, 2, 4, 8, 16, 32\}$. The workers are identical. Windows contain one request. Processing times lie in $[0.2, 1.0]$. The generation cap is 1,000 nodes. No retained tree reaches the cap.

Four node example. The fixed graph has edges $(0, 2)$, $(1, 2)$, and $(2, 3)$. Two FIFO workers process the graph. Each node has independent transient failure probability 0.15. There is no persistent fault in this comparison. We run 10,000 trials with no retry and with at most two retries per node. No trial reaches the global work cap.

C.2 Scheduling

All policies use the same realized request graphs and resource constraints. The CPU study uses 720 inputs feasible for all three policies. Every returned schedule is checked again for release times, precedence, memory, and worker order.

Client replay uses 72 inputs feasible for all three policies. The source graphs come from the controlled live study. Episodes with a cached root are excluded. Durations are observed on the client. Resource costs are normalized. These results do not measure GPU throughput or cache performance.

The exact comparison uses 20 five request instances with two workers. Only completed enumeration searches enter the reported optimality gaps.

C.3 Validation order

The live check uses three templates. They read a numeric field, sum a list, or count true flags. Each root feeds either one child or three children. The one child rule is $2x$. The three child rules are $2x$, $x + 3$, and $x - 1$. A final join sums the child outputs.

Two independent first root calls are used for each template and branch count. This gives 12 paired units. Each unit runs gate and speculation with an unmodified root and with one injected root fault. The full design has 48 logical episodes.

Within a paired unit, the first unmodified root response is shared across the four conditions. Every suffix is executed through the model API. The fault adds one to a correct root output exactly once. Gate validates the root before releasing its children. Speculation executes the full direct child layer before the root check. A rejected root invalidates those child executions. No join runs while the root is invalid.

The CPU validation scan uses correctness probability 0.6 and child mean 0.8. It tests

$$d \in \{0, 0.25, 0.49, 0.60, 0.90\}$$

with 50,000 roots per setting. Attempt caps are 10^2, 10^3, and 10^4. The same root streams are reused across caps. Capped runs are treated as censored.

Both live orders use complete local checks with $d = 1$. They use the same messages and the same caps of 12 node attempts and 180 seconds. The model is `qwen3.8-27b` with temperature 0.2 and top $p = 1$. The output cap is 256 tokens. One client channel spaces dispatches by at least 6.5 seconds. No episode reaches either cap. All 48 final workflows pass the final checks. Transport retries count as physical API requests but not as logical node executions.

C.4 Task recovery

The DataAgentBench study uses 18 tasks from 12 datasets [6]. The reported cohort was drawn from an original 54 task frame after the study scope was reduced for runtime. Some earlier corrected outcomes had already been observed. The final sampling rule uses task attributes only. It does not use correctness, runtime, cost, or rescue outcomes.

The sampling seed is 20261002. Every dataset contributes one task. Six datasets drawn without replacement contribute a second task. Tasks are then drawn within dataset strata. The final task list and source hashes were fixed before the new cohort calls. There is one run per selected task.

Earlier collections with execution boundary defects are excluded. A corrected partial collection completed before the scope change is also not pooled with the reported cohort.

The bootstrap resamples tasks within dataset strata and keeps the observed stratum sizes. Six strata contain one task and stay fixed in every resample. The reported ranges therefore condition on this selected sample. They do not measure variation across model seeds or over the full benchmark.

The three policies share one initial execution. Fresh restart begins a new execution after rejection. Stateful repair keeps the dialogue and execution state. Both recovery policies receive the same generic rejection signal. Each added phase allows 100 iterations and 900 seconds. The model is `qwen3.8-27b` with temperature 0.2, top $p = 1$, and an output cap of 2,048 tokens. One client channel spaces dispatches by at least 6.5 seconds.

Each logical policy includes the shared initial execution in its cost. The physical ledger charges it once. Missing provider usage stays missing. Recorded token totals are lower bounds when usage is incomplete. Timeouts, budget exhaustion, infrastructure failures, and validated wrong answers remain separate outcomes.

The client environment uses Windows on AMD64 with Python 3.11.6. Task execution uses isolated containers.

Software and tools. The extension contains 12 general code issues, 12 database and ORM issues, and 12 ToolSandbox scenarios. The software issues come from SWE-bench Verified [7]. The tool scenarios come from

ToolSandbox [4]. The cohort is fixed and purposive. It is not a benchmark wide sample.

Four tool scenarios that required an unavailable external API were replaced before tool collection. After some tool results had been observed, the planned database group was replaced by 12 fixed database and ORM issues with public tests. No model output for this replacement group existed when it was selected. The database and ORM group contains Django issues.

Software phases allow 100 model requests and 900 seconds. ToolSandbox also limits each phase to 30 new messages. Recovery order uses random seed. Software patches are checked in a separate evaluation container with the official SWE-bench v4.1.0 harness. ToolSandbox keeps its official continuous score and strict full credit as separate outcomes. Initial execution is shared physically and included in every logical policy cost. There is one policy comparison per task. Missing usage makes recorded token totals lower bounds. These results describe the selected tasks and do not measure repeated seed stability.